

I. scoring schemes

a) simple/unit scoring

δ	-	A	C	G	T
-	0	0	0	0	0
A	0	1	0	0	0
C	0	0	1	0	0
G	0	0	0	1	0
T	0	0	0	0	1

y. 0 gap penalty

A	T	T	A	-	C	G
A	-	T	A	T	C	G
1 +	0 +	1 +	1 +	0 +	1 +	1 = 5

vertical dash for identical seq

b) BLOSUM matrix: amended version of PAM

y.

Organism 1	B	A	B	A
Organism 2	A	A	A	C
Organism 3	A	A	C	C
Organism 4	A	A	B	A
Organism 5	A	A	C	C
Organism 6	A	A	B	C

total # = $6 \times 4 = 24$ aa

freq by letter pairs of letters aligned.

A $\frac{14}{24}$
B $\frac{4}{24}$
C $\frac{6}{24}$

$4 \times \binom{6}{2} = \frac{4 \times 6 \times 5}{2} = 60$
column 6 choose 2

using freq by letter observed

rule of independence

generate random seq & align them

Aligned Pair	Observed frequency	Expected Frequency	f
A to A	26/60	196/576	0.70
A to B	8/60	112/576	-1.09
A to C	10/60	168/576	-1.61
B to B	3/60	16/576	1.70
B to C	6/60	48/576	0.53
C to C	7/60	36/576	1.80

↑
hypothesis testing
estimated likelihood ratio
 $2 \cdot \log_2 \frac{\text{observed}}{\text{expected}}$

22. edit graph $G_{X,Y} = (V, E)$

a) def:

#1 notation:

$G_{X,Y}$: built from seq $X, Y \rightarrow \begin{matrix} X = x_1, \dots, x_m \\ Y = y_1, \dots, y_n \end{matrix}$

V : vertex set

E : edge set

#2. structural prop: DAG

Directed: forward direction

Acyclic: CAN'T revisit edges

Graph

b) vertices: pairwise alignments

#1. start from $(0,0)$

#2. # of vertices: $(m+1) \times (n+1)$

grid size $(m+1) \times (n+1)$

#3. grid pos:

$1 \leq i \leq m$

$1 \leq j \leq n$

c) edges: path

#1 weights assigned to edge: score

#2. 3 moves to a vertex

$\hookrightarrow$ possible edges $\hookrightarrow$ current loc

direction alignment edge: diagonal $\searrow$

notation $(i-1, j-1) \rightarrow (i, j)$

i, j increase

alignment $x_i \leftarrow$ consumed $y_j \leftarrow$ consumed

gap in X

horizontal $\rightarrow$

notation $(i, j-1) \rightarrow (i, j)$

j increase

$- \leftarrow$ No movement to next row $y_j \leftarrow$ consumed

gap in Y

vertical $\downarrow$

notation $(i-1, j) \rightarrow (i, j)$

i increases

$x_i \leftarrow$ consumed $- \leftarrow$ No movement to next column

d) optimization prob: best alignment $\xrightarrow[\text{[} \Sigma \text{ edge scores }]]{\text{scoring scheme}}$ highest scoring paths

#1. compute score for every possible path

runtime: exponential **BAD**

#2. solvency: dynamic programming

1' principle applied $\leftarrow$

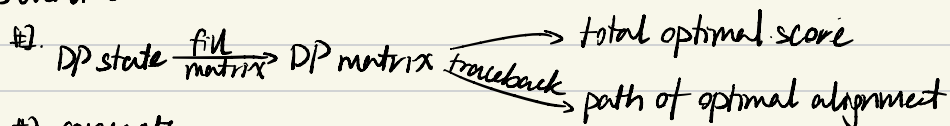
principle of optimality: optimal answer to a problem = optimal answers for its sub-problems

2' runtime: $O(mn)$

if $m=n \rightarrow O(n^2)$ quadratic

III. dynamic programming: solve larger problems using stored solutions to smaller subproblems.

a) overview



#2. concepts

1' DP state: one subproblem with stored optimal score

recurrence: formula for computing a cur state using values from earlier states

2' DP matrix

matrix filling: compute states in dependency order

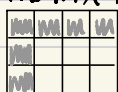
every state you need to compute our state must be already computed

3' traceback: starting from the final DP state & following the choices backward to reconstruct the path of optimal alignment.

b) global alignment

#1. define DP state: $S(i, j) \rightarrow$ score of best path from $(0, 0)$ to (i, j)

#2. matrix initialization: first row & column filled $\xrightarrow{\text{start}}$ $\xrightarrow{\text{current vertex}}$



set up all boundary cells

#3. filling matrix

1' val for DP state $\rightarrow$ recurrence
 at least 1 indices smaller

$$S(i, j) = \max \begin{cases} S(i-1, j-1) + \delta(x_i, y_j) \\ S(i-1, j) + \delta(x_i, -) \\ S(i, j-1) + \delta(-, y_j) \end{cases}$$

scores from predecessor states paths with assigned score

2' dependency order: column by column



pseudocode

```

1: function GLOBAL ALIGNMENT( $X \in \Sigma^m, Y \in \Sigma^n$ )
2:    $S_{0,0} \leftarrow 0$ 
3:   for  $i \in \{1, 2, \dots, m\}$  do
4:      $S_{i,0} \leftarrow S_{i-1,0} + \delta(X_i, -)$ 
5:   end for
6:   for  $j \in \{1, 2, \dots, n\}$  do
7:      $S_{0,j} \leftarrow S_{0,j-1} + \delta(-, Y_j)$ 
8:     for  $i \in \{1, 2, \dots, m\}$  do
9:        $S_{i,j} \leftarrow \max \begin{cases} S_{i-1,j-1} + \delta(X_i, Y_j) \\ S_{i-1,j} + \delta(X_i, -) \\ S_{i,j-1} + \delta(-, Y_j) \end{cases}$ 
10:    end for
11:  end for
12:  return  $S_{m,n}$ 
13: end function
  
```